

بِسْمِ اللَّهِ الرَّحْمَنِ الرَّحِيمِ

دوره میکروپایتون مقدماتی

بررسی کاربردی و تخصصی ساخت سیستم‌های هوشمند و نظارتی

مدرس : مهندس یاسین سموعی

🎯 برنامه‌نویس *IoT* | توسعه‌دهنده *Back-End*

🎓 کارشناسی ارشد الکترونیک دیجیتال

❄️ آذر ماه ۱۴۰۴

🏢 دانشگاه آزاد اسلامی واحد اصفهان



برنامه‌نویسی چیست و پایتون چه نقشی دارد؟

◈ برنامه‌نویسی چیست؟

برنامه‌نویسی یعنی طراحی و نوشتن مجموعه‌ای از دستورها که یک کامپیوتر، برد الکترونیکی یا هر دستگاه هوشمند بتواند آن‌ها را مرحله‌به‌مرحله اجرا کند.

با برنامه‌نویسی می‌توان رفتار یک سیستم را کنترل کرد، داده‌ها را پردازش کرد، عملیات‌های پیچیده را خودکار کرد و ابزارها و سرویس‌های جدید ساخت. در واقع برنامه‌نویسی مهارتی است که ایده را به یک محصول قابل اجرا تبدیل می‌کند.

◈ پایتون چیست؟

پایتون یک زبان برنامه‌نویسی سطح بالا، خوانا و بسیار کاربردی است که برای سادگی یادگیری طراحی شده.

با وجود سادگی ظاهری، پایتون بسیار قدرتمند است و در حوزه‌های مختلف مثل توسعه وب، تحلیل داده، هوش مصنوعی، اتوماسیون، رباتیک و مخصوصاً اینترنت اشیا استفاده می‌شود.

کتابخانه‌های گسترده، جامعه کاربری بزرگ و پشتیبانی از پلتفرم‌های مختلف باعث شده پایتون به یکی از محبوب‌ترین ابزارهای برنامه‌نویسان تبدیل شود.

تفاوت میکروپایتون با پایتون

♦ پایتون :

- یک زبان برنامه‌نویسی کامل و قدرتمند برای کامپیوترها و سرورهاست.
- دارای کتابخانه‌های بسیار زیاد برای وب، هوش مصنوعی، تحلیل داده و...
- روی سیستم‌عامل اجرا می‌شود (**Windows** , **Linux** , **macOS**)
- مصرف حافظه و پردازنده بیشتر است چون برای سخت‌افزارهای قوی طراحی شده.

♦ میکروپایتون :

- نسخه سبک و بهینه‌شده پایتون برای میکروکنترلرها مثل **ESP32**
- حجم کد بسیار کمتر است و فقط امکانات ضروری پایتون را دارد.
- به‌جای سیستم‌عامل، مستقیم روی سخت‌افزار اجرا می‌شود.
- دسترسی داخلی به پایه‌ها، سنسورها، تایمرها و پروتکل‌هایی مثل **I2C/SPI/UART** دارد.
- برای دستگاه‌هایی با **RAM** و حافظه فلش بسیار کم ساخته شده مثلاً چند ده کیلوبایت.



تفاوت میکروپایتون با آردوینو



◇ زبان برنامه نویسی

:MicroPython

از پایتون استفاده می کند؛ خواناتر، ساده تر و مناسب توسعه سریع.

:Arduino

از C/C++ استفاده می کند؛ سریع تر، سطح پایین تر و کنترل دقیق تر روی سخت افزار می دهد.

◇ سطح دسترسی و مدل اجرا

:MicroPython

یک مفسر روی برد اجرا می شود؛ کدها به صورت تعاملی و سریع تست می شوند. (REPL)

:Arduino:

کد به زبان ماشین کامپایل می شود و مستقیماً روی MCU اجرا می شود؛ بدون لایه ی مفسر.

◇ سرعت و عملکرد

:MicroPython

سرعت کمتر به خاطر مفسر بودن، مناسب پروژه های IoT و کنترل های معمولی.

:Arduino

بسیار سریع تر؛ مناسب کارهای real-time و پروژه های حساس و دقیق، مثل: کنترل موتور، سنسورهای بدون تأخیر.

REPL چیست؟

REPL اصطلاحی است برگرفته از واژگان **Read Evaluate Print Loop** که به منزله یک محیط برنامه‌نویسی تعاملی

است که از کاربر ورودی می‌گیرد، آن را ارزیابی کرده و بعد نتیجه را به وی باز می‌گرداند به طوری که دیگر نیازی به صرف زمان برای

کامپایل و منتظر ماندن نیست و این در حالی است که برنامه نوشته شده در محیط‌های مبتنی بر **REPL** به صورت اصطلاحاً

Piecewise (بخش به بخش) اجرا می‌شود.

```
Command Prompt - python
Microsoft Windows [Version 10.0.26200.7171]
(c) Microsoft Corporation. All rights reserved.

C:\Users\user>python
Python 3.11.4 (tags/v3.11.4:d2340ef, Jun  7 2023, 05:45:37) [MSC v.1934 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license" for more information.
>>> print("Hello World")
Hello World
>>> |
```


مفهوم مفسر و کامپایلر

۱. مفسر (Interpreter) چیست؟

مفسر مثل یک مترجم هم‌زمان است.

یعنی دستورات برنامه را خط‌به‌خط می‌خواند و همان لحظه اجرا می‌کند.

مثال روزمره:

فرض کن شما به یک مترجم می‌گید:

«جمله اول رو ترجمه کن».

بعد می‌گید «حالا جمله دوم».

مترجم هر جمله رو همان لحظه ترجمه و اجرا می‌کند.

مفسر = اجرا لحظه‌ای و خط‌به‌خط

۲. کامپایلر (Compiler) چیست؟

کامپایلر مثل یک مترجم کتاب است.

کل برنامه را یک‌جا می‌گیرد، آن را تبدیل به یک نسخه قابل اجرا برای دستگاه می‌کند، و بعد آن نسخه اجرا می‌شود.

مثال روزمره:

شما یک کتاب فارسی به مترجم می‌دهید.

مترجم کل کتاب را تبدیل به یک کتاب انگلیسی می‌کند.

بعد از آن، بدون نیاز به مترجم، می‌توانید کتاب ترجمه‌شده را بخوانید.

کامپایلر = ترجمه کامل قبل از اجرا

حالا این‌ها چه ربطی به **MicroPython** و **Arduino** دارند؟

MicroPython : مفسری

شما یک دستور **MicroPython** می‌نویسید

مفسر **MicroPython** که داخل برد هست همان لحظه اجرا می‌کند.

نیاز نیست کل برنامه یک‌بار کامل ترجمه شود.

تست و اصلاح (دیباگ) سریع‌تر است.

Arduino : کامپایلری

یعنی چه؟

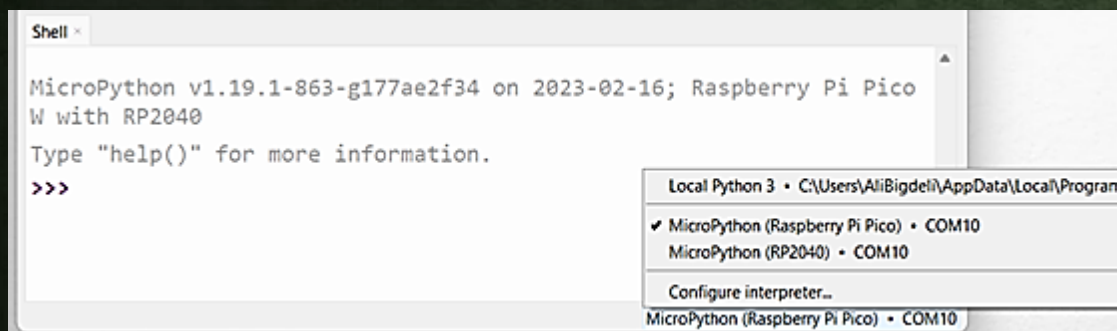
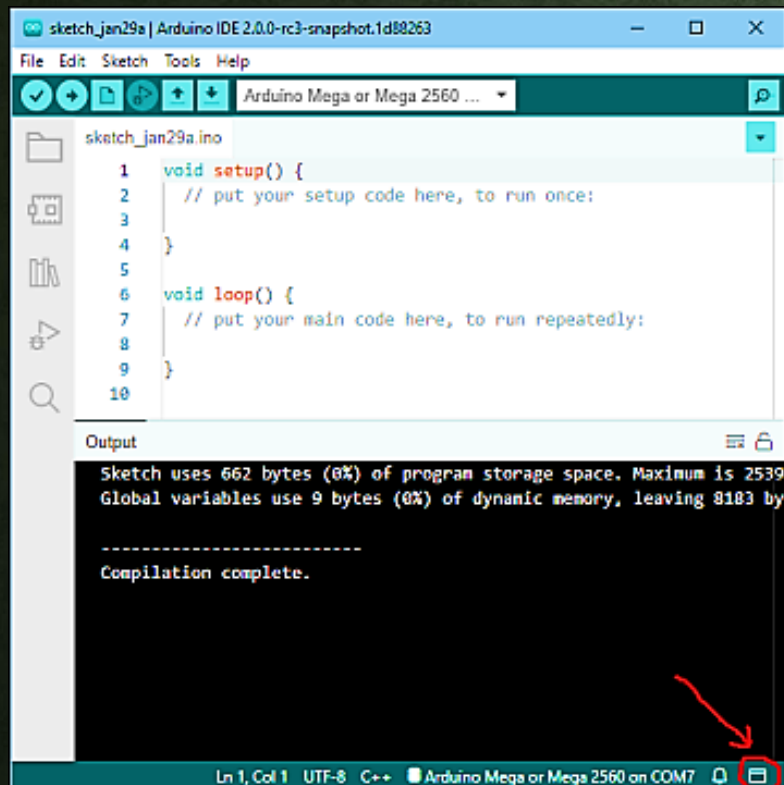
شما کد را می‌نویسید.

متن کامل برنامه با دکمه **Upload** کامپایل می‌شود.

تبدیل می‌شود به زبان قابل فهم برای میکروکنترلر (کد ماشین).

بعد روی برد آپلود می‌شود.

سپس اجرا می‌شود.



آشنایی با میکروکنترلرها و تفاوت با میکروپروسسور

میکروکنترلر (MCU) چیست؟

میکروکنترلر یک کامپیوتر کوچک و مستقل روی یک چیپ است که داخل خودش همه چیز را دارد:

پردازنده (CPU)

حافظه RAM

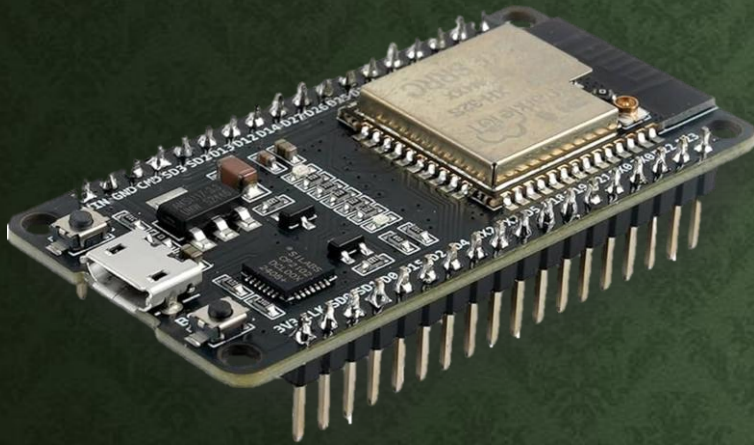
حافظه برنامه (Flash)

ورودی/خروجی‌ها (GPIO)

تایمر، ADC، DAC

پروتکل‌های ارتباطی UART، SPI، I2C

یعنی میکروکنترلر تمام قطعات لازم برای کنترل یک دستگاه را یک جا دارد.



میکروپروسسور (Microprocessor) چیست؟

میکروپروسسور فقط یک واحد پردازشی مثل CPU است.

یعنی برخلاف میکروکنترلر:

✗ RAM ندارد

✗ Flash ندارد

✗ GPIO و ADC ... ندارد

✗ مستقل نیست

برای کار کردن نیاز دارد کنار خودش قطعات زیادی اضافه شود.

مثلاً CPU لپ‌تاپ یا Raspberry Pi همین‌طورند.



آشنایی با میکروکنترلر های پر کاربرد اینترنت اشیا

ویژگی	Raspberry Pi Pico	ESP32	ESP8266
نوع پردازنده	تک هسته‌ای، ARM Cortex-M0+	دو هسته‌ای، ۳۲-bit	تک هسته‌ای، ۳۲-bit
فرکانس CPU	133 MHz	160–240 MHz	80–160 MHz
RAM داخلی	264 KB	520 KB	~50 KB
حافظه فلش	2 MB	تا ۱۶ MB	تا ۱۶ MB
Wi-Fi	✗ ندارد	✓ دارد	✓ دارد
بلوتوث	✗ ندارد	BLE (Bluetooth Low Energy)	✗ ندارد
تعداد GPIO	26	~34	~17
ADC / DAC	3 ADC (12-bit)	چند ADC (12-bit) + DAC	1 ADC (10-bit)
PWM / I2C / SPI / UART	✓	✓ پیشرفته‌تر	✓
قیمت تقریبی	کم	متوسط	کم
کاربرد پیشنهادی	پروژه‌های آموزش، کنترل سخت‌افزار، IoT ساده با USB	IoT پیشرفته، رباتیک، BLE، Wi-Fi	پروژه‌های ساده IoT با Wi-Fi
مزیت کلیدی	سادگی، USB مستقیم، ارزان، پردازنده ARM	قدرتمند، چندمنظوره، Wi-Fi + BLE	کوچک، ارزان، وای‌فای داخلی